DIPLOMADO EN FULLSTACK DEVELOPER BACKEND FRONTEND

DESARROLLO BACKEND CON DJANGO

MÉTODO DE EVALUACIÓN

Asistencia	10%
Participación en clases	10%
Examen	40%
Proyecto	40%

TABLA DE CONTENIDOS

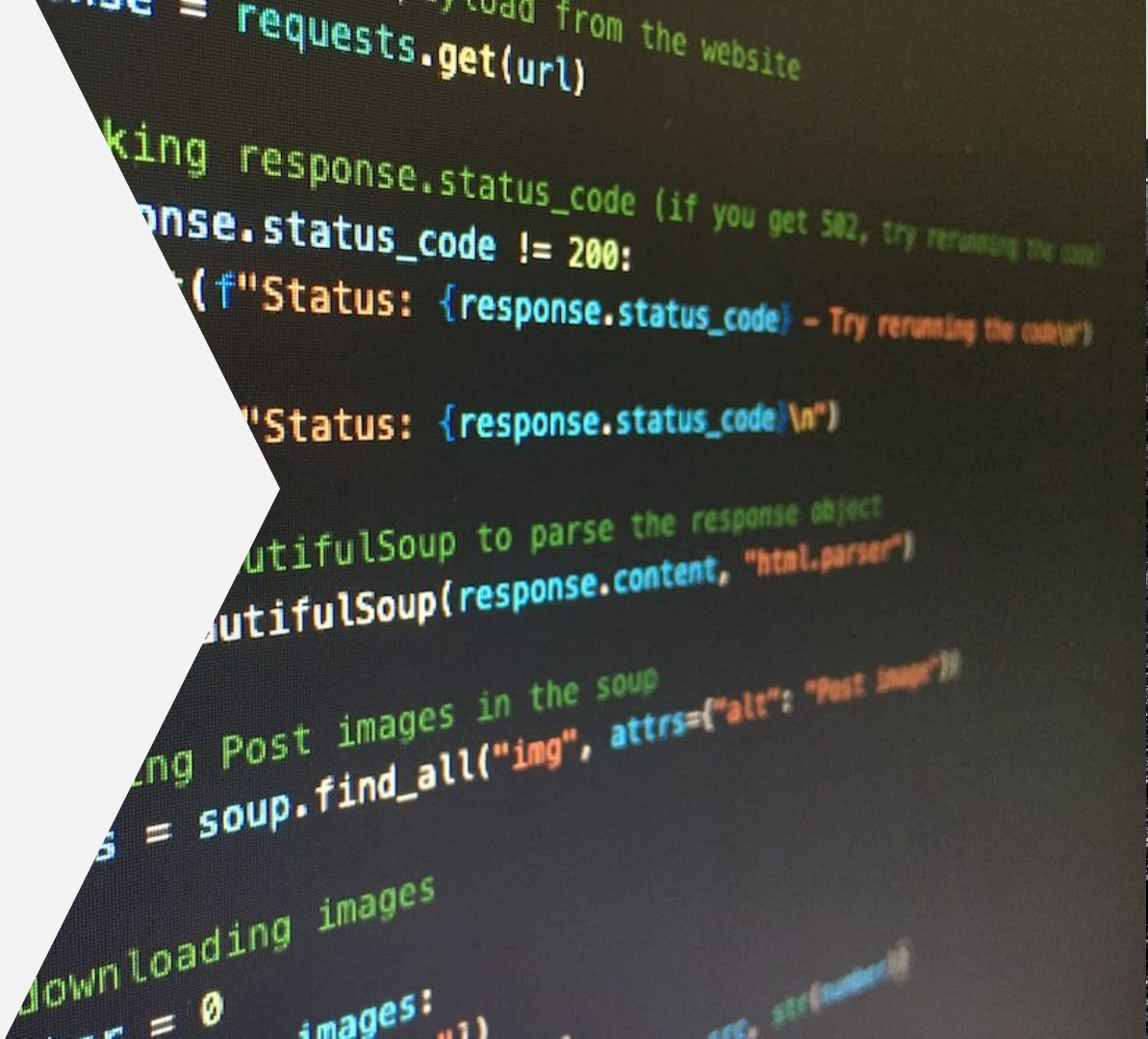
Estructura del Framework

Modelos y Administración

Django Rest Framework

Testing y Logs

Formularios



Trabajando con formularios

inventario/templates/form_categorias.html

```
<form action="{% url 'categorias' %}" method="post">
{% csrf_token %}
<fieldset>
    <legend><h1>Categoria</h1></legend>
    {% if error_message %}
    <p class="error">{{error_message}}</p>
    {% endif %}
    <label for="nombre">Nombre</label>
    <input type="text" name="nombre"/>
    <br/>
    <input type="submit" value="Enviar"/>
</fieldset>
</form>

<ul>
    {% for categoria in categorias %}
    <li>{{categoria}}</li>
    {% endfor %}
</ul>
```

Trabajando con formularios

inventario/views.py

```python
from django.http.response import HttpResponse
from django.shortcuts import render
from .models import Categoria


def index(request):
    return HttpResponse('Hola Mundo')

def contact(request, name):
    return HttpResponse(f"Bienvenido {name} a la clase de Django")

def categorias(request):
    post_nombre = request.POST.get('nombre')
    if post_nombre:
        q = Categoria(nombre=post_nombre)
        q.save()

    filtro_nombre = request.GET.get('nombre')
    if filtro_nombre:
        categorias = Categoria.objects.filter(nombre__contains=filtro_nombre)
    else:
        categorias = Categoria.objects.all()

    return render(request, "form_categorias.html", {
        "categorias": categorias
    })
```

Utilizando ModelForms

inventario/templates/form_productos.html

```
<h1>Producto</h1>

<form action="" method="post">
  {% csrf_token %}
  <table>
    {{ form.as_table }}
  </table>
  <input type="submit" value="Submit" />
</form>
```

Utilizando ModelForms

inventario/forms.py

```
from django import forms
from .models import Producto


class ProductoForm(forms.ModelForm):
    class Meta:
        model = Producto
        fields = "__all__"
```

Utilizando ModelForms

inventario/views.py

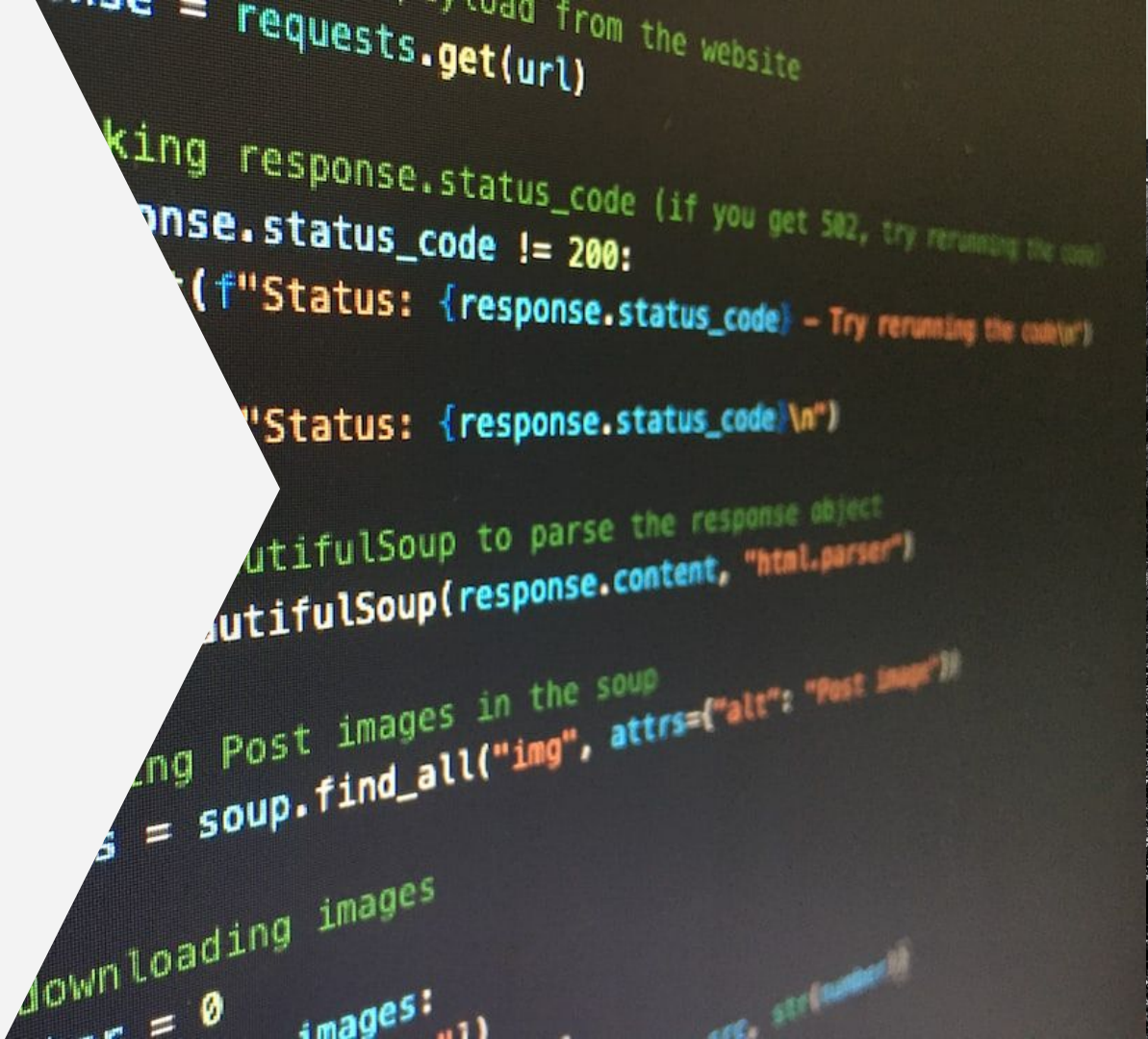
```
def productoFormView(request):
    form = ProductoForm(request.POST)
    # check if form data is valid
    if form.is_valid():
        # save the form data to model
        form.save()

    return render(request, "producto_form.html", {"form": form})
```

Utilizando ModelForms

inventario/views.py

```python
from django.shortcuts import get_object_or_404


def productoFormView(request):
    form = ProductoForm()
    producto = None
    id_producto = request.GET.get('id')
    if id_producto:
        producto = get_object_or_404(Producto, id=id_producto)
        form = ProductoForm(instance=producto)

    if request.method == 'POST':
        if producto:
            form = ProductoForm(request.POST, instance=producto)
        else:
            form = ProductoForm(request.POST)

    if form.is_valid():
        form.save()

    return render(request, "producto_form.html", {"form": form})
```

Validadores



Custom Validator

inventario/validators.py

```python
from django.core.exceptions import ValidationError


def validar_par(value):
    if value % 2 != 0:
        raise ValidationError(
            '%(value)s no es un numero par',
            params={'value': value},
        )
```

Aplicando validadores

inventario/models.py

```
from .validators import validar_par
```

```
class Producto(models.Model):
    ...
    precio = models.DecimalField(decimal_places=2,
              max_digits=10, validators=[validar_par,])
    ...
```

Validadores standard

inventario/models.py

```
from django.core.validators import EmailValidator
```

```
class Persona(models.Model):
    ...
    email = models.CharField(max_length=100, unique=True,
              validators=[EmailValidator('No es un email valido')])
    ...
```

https://docs.djangoproject.com/en/6.0/ref/validators/

https://docs.djangoproject.com/en/6.0/ref/models/instances/#validating-objects

Tests

requests.get(url)
response.status_code != 200:
Status: {response.status_code}
Status: {response.status_code}\n
utifulSoup to parse the response object
utifulSoup(response.content, "html.parser")
ng Post images in the soup
= soup.find_all("img", attrs={"alt": "Post image"})
downloading images
images:

¿Qué son los tests?

Los tests son funciones que nos ayudan a que nuestro código opere correctamente.

¿Para qué hacer tests?

1. Encontrar errores de código:Cuando tu código crece en cientos de líneas o incluso miles, el que sufra un error puede ser catastrófico (puede llevar varios días solucionarlo e incluso semanas). Sirve para encontrar errores futuros que a simple vista no seríamos capaces de ver (son funciones que trabajan sobre nuestras funciones principales de código).
2. Nos inclina a ser profesionales:Los tests le dan un aspecto profesional e incita a que incluso las personas expertas quieran contribuir en nuestro proyecto (la demanda de desarrollo crece aún más y la oferta sigue estancada en un punto).Nos permite trabajar mejor en equipo, ya que es muy probable que cuando se lea nuestro código no se entienda por completo y que simplemente se haga una contribución que se crea pertinente (así sabe si la contribución fue adecuada).

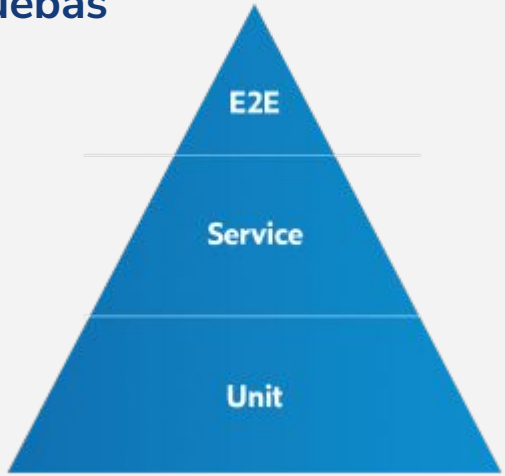
TDD o Test-Driven Development

(Desarrollo dirigido por tests) Es una práctica de programación que consiste en escribir primero las pruebas (generalmente unitarias), después escribir el código fuente que pase la prueba satisfactoriamente y, por último, refactorizar el código escrito.

Pasos a seguir en la realización de tests:

1. Identificar el problema
2. Crear el test
3. Correr el test
4. Arreglar el problema
5. Correr el test

Automatización de Pruebas



Las pruebas unitarias están en la parte inferior y representan las pruebas más granulares, de las cuales debería tener muchas.

La siguiente capa en la pirámide tiene capa de servicio o pruebas de integración. Aquí es donde comienza a probar cómo interactúan sus componentes de software entre sí, ya sean integraciones internas o externas.

La prueba de extremo a extremo (E2E) es el más complejo y prueba el software en su conjunto para asegurarse de que funciona como se espera de principio a fin.

Estructura

```python
from django.test import TestCase
from django.urls.base import reverse
from .models import Categoria


class TestCategorias(TestCase):
    # Fixtures cargados para test
    fixtures = ['fixture1.json', 'fixture2.json', ...]

    # Configuraciones globales
    def setUp(self):
      ...

    # Test 1
    def test_metodo1(self):
      ...

    # Test 2
    def test_metodo2(self):
      ...
```

Trabajando con Test Unitarios

inventario/tests.py

```
from django.test import TestCase
from .models import Categoria


class TestCategorias(TestCase):
    def test_grabacion(self):
        q = Categoria(nombre="Bebidas")
        q.save()
        self.assertEqual(Categoria.objects.count(), 1)
```

Ejecutar los tests

```
python manage.py test inventario
```

Tipos de assert

Method	Checks that	New in
`assertEqual(a, b)`	`a == b`	
`assertNotEqual(a, b)`	`a != b`	
`assertTrue(x)`	`bool(x) is True`	
`assertFalse(x)`	`bool(x) is False`	
`assertIs(a, b)`	`a is b`	3.1
`assertIsNot(a, b)`	`a is not b`	3.1
`assertIsNone(x)`	`x is None`	3.1
`assertIsNotNone(x)`	`x is not None`	3.1
`assertIn(a, b)`	`a in b`	3.1
`assertNotIn(a, b)`	`a not in b`	3.1
`assertIsInstance(a, b)`	`isinstance(a, b)`	3.2
`assertNotIsInstance(a, b)`	`not isinstance(a, b)`	3.2

https://docs.djangoproject.com/en/6.0/topics/testing/tools/#assertions

Trabajando con Validaciones en Tests

Agregar el siguiente validador en el atributo nombre del modelo Categoría

inventario/validators.py

```
def validation_categoria(value):
    if value == 'No permitido':
        raise ValidationError("No es una opcion permitida")
```

Trabajando con Validaciones en Tests

inventario/tests.py

```
def test_grabacion(self):
        q = Categoria.objects.create(nombre="No permitido")
        self.assertRaises(ValidationError, q.full_clean)
```

Trabajando con Validaciones en Tests

inventario/tests.py

```
from django.core.exceptions import ValidationError
```

```
def test_grabacion(self):
        with self.assertRaises(ValidationError) as qv:
            q = Categoria.objects.create(nombre="No permitido")
            q.full_clean()

        mensaje_error = dict(qv.exception)
        self.assertEqual(mensaje_error["nombre"][0], "No es una opcion permitida")
```

Trabajando con Test de Integración

inventario/tests.py

```
from django.test import Client
```

```
def setUp(self):
        self.client = Client()
        Categoria.objects.create(nombre="Categoria 1")
        Categoria.objects.create(nombre="Categoria 2")
```

```
def test_categoria_list(self):
        response = self.client.get('/inventario/categorias')
        self.assertContains(response, "Categoria 1", status_code=200, html=True)

    def test_categoria_filter(self):
        response = self.client.get('/inventario/categorias?nombre=Categoria 2')
        self.assertNotContains(response, "Categoria 1", status_code=200, html=True)

    def test_categoria_post(self):
        response = self.client.post('/inventario/categorias', {"nombre": "Categoria
        self.assertContains(response, "Categoria 3", status_code=200, html=True)
```

Test tags

```python
from django.test import tag

class SampleTestCase(TestCase):

    @tag('fast')
    def test_fast(self):
        ...

    @tag('slow')
    def test_slow(self):
        ...

    @tag('slow', 'core')
    def test_slow_but_core(self):
```

```
python manage.py test --tag=fast
```

GRACIAS